

```

# -*- coding: utf-8 -*-
import calendar
from datetime import datetime, timedelta
import json
import os
import re
import threading
import time
import requests
from requests.adapters import HTTPAdapter

TELEGRAM_TOKEN = "8941143336:AAFDMxxvGS3W_eelaaulNiF_SiH4n9FoVYw"
TRAVELPAYOUTS_TOKEN = "238e7accff4bacc901840d3d8ef25c85"
URL = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}"

# ===== СКОРОСТНЫЕ ПУЛЫ СОЕДИНЕНИЙ =====
adapter = HTTPAdapter(pool_connections=15, pool_maxsize=30, max_retries=1)
tg_session = requests.Session()
tg_session.mount("https://", adapter)
tp_session = requests.Session()
tp_session.mount("https://", adapter)

# ===== ХРАНЕНИЕ ПОДПИСОК И КЭШ =====
def get_subs_file_path():
    try:
        script_dir = os.path.dirname(os.path.abspath(__file__))
        if script_dir and os.path.exists(script_dir):
            return os.path.join(script_dir, "subscriptions.json")
    except Exception:
        pass
    for folder in [
        "/storage/emulated/0/Documents",
        "/storage/emulated/0/Download",
        os.getcwd(),
    ]:
        try:
            if os.path.exists(folder):
                return os.path.join(folder, "subscriptions.json")
        except Exception:
            pass
    return "subscriptions.json"

SUBS_FILE = get_subs_file_path()

```

```
PRICES_CACHE = {}  
CACHE_TTL = 300 # 5 минут
```

```
MONTHS_RU = [  
    "", "Январь", "Февраль", "Март", "Апрель", "Май", "Июнь",  
    "Июль", "Август", "Сентябрь", "Октябрь", "Ноябрь", "Декабрь"  
]
```

```
MONTHS_SHORT_MAP = {  
    "янв": 1, "фев": 2, "мар": 3, "апр": 4, "май": 5, "мая": 5,  
    "июн": 6, "июл": 7, "авг": 8, "сен": 9, "окт": 10, "ноя": 11, "дек": 12  
}
```

```
POPULAR_FROM_CITIES = [  
    ("Москва", "MOW"),  
    ("Санкт-Петербург", "LED"),  
    ("Казань", "KZN"),  
    ("Екатеринбург", "SVX"),  
    ("Новосибирск", "OVB"),  
    ("Самара", "KUF"),  
    ("Уфа", "UFA"),  
    ("Сочи", "AER"),  
    ("Красноярск", "KJA"),  
    ("Мин. Воды", "MRV"),  
]
```

```
POPULAR_TO_CITIES = [  
    ("Сочи", "AER"),  
    ("Стамбул", "IST"),  
    ("Дубай", "DXB"),  
    ("Пхукет", "HKT"),  
    ("Калининград", "KGD"),  
    ("Анталья", "AYT"),  
    ("Ереван", "EVN"),  
    ("Баку", "GYD"),  
    ("Ташкент", "TAS"),  
    ("Бангкок", "BKK"),  
]
```

```
CITY_NAMES = {  
    "MOW": "Москва", "SVO": "Москва", "DME": "Москва", "VKO": "Москва", "ZIA": "Москва",  
    "LED": "Санкт-Петербург", "KZN": "Казань", "SVX": "Екатеринбург", "OVB": "Новосибирск",  
    "KUF": "Самара", "UFA": "Уфа", "AER": "Сочи", "KJA": "Красноярск", "MRV":
```

"Минеральные Воды",
 "IST": "Стамбул", "SAW": "Стамбул", "DXB": "Дубай", "DWC": "Дубай", "SHJ": "Шарджа",
 "HKT": "Пхукет", "BKK": "Бангкок", "DMK": "Бангкок", "UTP": "Паттайя",
 "KGD": "Калининград", "AYT": "Анталья", "EVN": "Ереван", "GYD": "Баку",
 "TAS": "Ташкент", "NQZ": "Астана", "ALA": "Алматы", "FRU": "Бишкек", "DYU": "Душанбе",
 "TBS": "Тбилиси", "BUS": "Батуми", "GOI": "Гоа", "MLE": "Мале", "CMB": "Коломбо",
 "HRG": "Хургада", "SSH": "Шарм-эль-Шейх", "CAI": "Каир", "DOH": "Доха",
 "ROV": "Ростов-на-Дону", "VOG": "Волгоград", "GOJ": "Нижний Новгород",
 "PEE": "Пермь", "CEK": "Челябинск", "OMS": "Омск", "TOF": "Томск",
 "IKT": "Иркутск", "KHV": "Хабаровск", "VVO": "Владивосток", "YKS": "Якутск",
 "PKC": "Петропавловск-Камчатский", "UUS": "Южно-Сахалинск", "GDH": "Магадан",
 "MMK": "Мурманск", "ARH": "Архангельск", "SCW": "Сыктывкар", "KRR": "Краснодар",
 "AAQ": "Анапа", "MCX": "Махачкала", "GRV": "Грозный", "OGZ": "Владикавказ",
 "NAL": "Нальчик", "STW": "Ставрополь", "ASF": "Астрахань", "REN": "Оренбург",
 "TJM": "Тюмень", "SGC": "Сургут", "HMA": "Ханты-Мансийск", "NOJ": "Ноябрьск",
 "NUX": "Новый Уренгой", "CSY": "Чебоксары", "IJK": "Ижевск", "NBC": "Нижекамск",
 "SKX": "Саранск", "BZK": "Брянск", "VOZ": "Воронеж", "TBW": "Тамбов", "EGO":
 "Белгород",
 "URS": "Курск", "KLF": "Калуга", "IWA": "Иваново", "PES": "Петрозаводск"
 }

AIRLINES = {
 "DP": "Победа",
 "SU": "Аэрофлот",
 "FV": "Россия",
 "S7": "S7 Airlines",
 "U6": "Уральские авиалинии",
 "UT": "ЮТэйр (Utair)",
 "A4": "Азимут",
 "WZ": "Red Wings",
 "5N": "Smartavia",
 "N4": "Северный Ветер (Nordwind)",
 "EO": "Икар (Pegas Fly)",
 "RT": "ЮВТ Аэро",
 "7R": "РусЛайн",
 "Y7": "Нордстар (NordStar)",
 "IO": "ИрАэро",
 "D2": "Северсталь Авиа",
 "R3": "Якутия",
 "HZ": "Аврора",
 "2G": "Ангара",
 "KV": "Красавиа",
 "I8": "Ижавиа",
 }

```

"TK": "Turkish Airlines",
"PC": "Pegasus Airlines",
"FZ": "Flydubai",
"EK": "Emirates",
"G9": "Air Arabia",
"QR": "Qatar Airways",
"EY": "Etihad Airways",
"J2": "Азербайджанские авиалинии (AZAL)",
"HY": "Узбекские авиалинии (Uzbekistan Airways)",
"KC": "Эйр Астана (Air Astana)",
"DV": "SCAT Airlines",
"B2": "Белавиа (Belavia)",
"3F": "FlyOne Armenia",
"RM": "Air Dilijans",
"W6": "Wizz Air",
"CZ": "China Southern Airlines",
"MU": "China Eastern Airlines",
"CA": "Air China",
}

```

```

TIME_FILTERS = {
    "any": {"label": "Любое время", "short": "Любое", "icon": "🕒", "hours": (0, 24)},
    "morning": {"label": "Утро (06:00 - 12:00)", "short": "Утро", "icon": "🌅", "hours": (6, 12)},
    "day": {"label": "День (12:00 - 18:00)", "short": "День", "icon": "☀️", "hours": (12, 18)},
    "evening": {"label": "Вечер (18:00 - 00:00)", "short": "Вечер", "icon": "🌙", "hours": (18, 24)},
}

```

```

user_sessions = {}

```

```

def load_subs():
    if os.path.exists(SUBS_FILE):
        try:
            with open(SUBS_FILE, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            pass
    return []

```

```

def save_subs(subs):
    try:
        with open(SUBS_FILE, "w", encoding="utf-8") as f:
            json.dump(subs, f, ensure_ascii=False, indent=2)
    except Exception:

```

pass

```
def get_session(chat_id):
    if chat_id not in user_sessions:
        user_sessions[chat_id] = {
            "from_code": None,
            "from_name": None,
            "to_code": None,
            "to_name": None,
            "mode": "main",
            "time_filter": "any",
            "cal_year": None,
            "cal_month": None,
        }
    return user_sessions[chat_id]
```

```
def get_city_name(code: str):
    if not code:
        return ""
    clean_code = code.upper().strip()
    if clean_code in CITY_NAMES:
        return CITY_NAMES[clean_code]
    c_code, c_name = find_city_smart(clean_code)
    if c_name:
        CITY_NAMES[clean_code] = c_name
    return c_name
    return clean_code
```

```
def get_airline_name(air_code: str, flight_number=None):
    if not air_code:
        return "Авиакомпания"
    code = air_code.upper().strip()
    name = AIRLINES.get(code)
    if not name:
        try:
            r = tp_session.get(
```

```
f"https://autocomplete.travelpayouts.com/places2?term={code}&locale=ru&types[]=airline",
            timeout=2
        )
    if r.status_code == 200:
        items = r.json()
        if items:
```

```

        name = items[0].get("name")
        if name:
            AIRLINES[code] = name
    except Exception:
        pass
    full_name = name or code
    if flight_number:
        return f"{{full_name}} (рейс {{code}}-{{flight_number}})"
    return full_name

def find_city_smart(name: str):
    clean = name.strip()
    if not clean:
        return None, None
    if len(clean) == 3 and clean.isalpha() and clean.isascii():
        return clean.upper(), clean.upper()
    try:
        r = tp_session.get(
            f"https://autocomplete.travelpayouts.com/places2?term={clean}&locale=ru&types[]=city",
            timeout=3
        )
        if r.status_code == 200:
            items = r.json()
            if items:
                first_item = items[0]
                return first_item.get("code"), first_item.get("name", clean.capitalize())
    except Exception:
        pass
    return None, None

def get_official_aviasales_prices(origin, dest, departure_month=None):
    cache_key = (origin, dest, departure_month or "")
    now = time.time()
    if cache_key in PRICES_CACHE:
        ts, data = PRICES_CACHE[cache_key]
        if now - ts < CACHE_TTL:
            return data
    api_url = "https://api.travelpayouts.com/aviasales/v3/prices_for_dates"
    headers = {"X-Access-Token": TRAVELPAYOUTS_TOKEN, "Accept": "application/json"}
    params = {
        "origin": origin,
        "destination": dest,
        "currency": "rub",

```

```

        "sorting": "price",
        "limit": 30,
        "token": TRAVELPAYOUTS_TOKEN
    }
    if departure_month:
        params['departure_at'] = departure_month
    try:
        r = tp_session.get(api_url, headers=headers, params=params, timeout=3)
        if r.status_code == 200:
            data = r.json().get("data", [])
            PRICES_CACHE[cache_key] = (now, data)
            return data
    except Exception:
        pass
    return []

```

```

def format_datetime_ru(date_str):
    try:
        clean_dt = date_str[:19]
        if "T" in clean_dt:
            dt = datetime.strptime(clean_dt, "%Y-%m-%dT%H:%M:%S")
            dep_time = dt.strftime("%H:%M")
        else:
            dt = datetime.strptime(clean_dt[:10], "%Y-%m-%d")
            dep_time = None
        months = [
            "января", "февраля", "марта", "апреля", "мая", "июня",
            "июля", "августа", "сентября", "октября", "ноября", "декабря"
        ]
        weekdays = ["Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Вс"]
        m_idx = dt.month - 1
        pretty_date = f"{dt.day} {months[m_idx]} ({weekdays[dt.weekday()]})"
        date_code = dt.strftime("%d%m")
        return pretty_date, date_code, dep_time
    except Exception:
        return date_str[:10], date_str[:10].replace("-", "").[4:8], None

```

```

def extract_date(text):
    now = datetime.now()
    m = re.search(r"(\d{1,2})[./](\d{1,2})(?:[./](\d{2,4}))? ", text)
    if m:
        day, month = int(m.group(1)), int(m.group(2))
        year = int(m.group(3)) if m.group(3) else now.year

```

```

    if year < 100:
        year += 2000
    try:
        d = datetime(year, month, day)
        if d < now - timedelta(days=1) and not m.group(3):
            d = datetime(year + 1, month, day)
        return d.strftime("%Y-%m-%d"), d.strftime("%d%m")
    except Exception:
        pass
for m_root, m_num in MONTHS_SHORT_MAP.items():
    m = re.search(rf" (\d{{1,2}})\s+{m_root}\w* ", text.lower())
    if m:
        day = int(m.group(1))
        month = m_num
        try:
            d = datetime(now.year, month, day)
            if d < now - timedelta(days=1):
                d = datetime(now.year + 1, month, day)
            return d.strftime("%Y-%m-%d"), d.strftime("%d%m")
        except Exception:
            pass
return None, None

def send_tg(chat_id, text, markup=None):
    payload = {
        "chat_id": chat_id,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": True
    }
    if markup:
        payload['reply_markup'] = markup
    try:
        return tg_session.post(f"{URL}/sendMessage", json=payload, timeout=6).json()
    except Exception:
        return {}

def edit_tg(chat_id, message_id, text, markup=None):
    payload = {
        "chat_id": chat_id,
        "message_id": message_id,
        "text": text,
        "parse_mode": "HTML",
    }

```

```

        "disable_web_page_preview": True
    }
    if markup:
        payload['reply_markup'] = markup
    try:
        return tg_session.post(f"{URL}/editMessageText", json=payload, timeout=6).json()
    except Exception:
        return {}

def answer_cq(cq_id, text=None):
    def _run():
        payload = {"callback_query_id": cq_id}
        if text:
            payload['text'] = text
        try:
            tg_session.post(f"{URL}/answerCallbackQuery", json=payload, timeout=3)
        except Exception:
            pass
    threading.Thread(target=_run, daemon=True).start()

# ===== ФИЗИЧЕСКАЯ КЛАВИАТУРА ВНИЗУ =====
def main_reply_keyboard():
    return {
        "keyboard": [
            [{"text": "✈️ Новый поиск"}],
            [{"text": "🔥 Топ-5 дешевых"}, {"text": "📋 Мои отслеживания"}],
            [{"text": "↩️ Сбросить выбор"}],
        ],
        "resize_keyboard": True,
        "is_persistent": True,
    }

# ===== ПОШАГОВЫЙ ИНТЕРАКТИВНЫЙ ПОИСК =====
def step1_from_panel():
    text = (
        "✈️ <b>Поиск билетов на Авиасейлс</b>\n\n"
        "<b>Шаг 1 из 3:</b> Выберите <b>город вылета (Откуда)</b> 📍"
    )
    keyboard = []
    row = []
    for name, code in POPULAR_FROM_CITIES:
        row.append({"text": name, "callback_data": f"wizard_from:{code}:{name}"})

```

```

        if len(row) == 2:
            keyboard.append(row)
            row = []
    if row:
        keyboard.append(row)
    keyboard.append([{"text": "✎ Ввести другой город вручную", "callback_data":
"wizard_type:from"}])
    return text, {"inline_keyboard": keyboard}

def step2_to_panel(sess):
    fn = sess.get('from_name', 'Город вылета')
    text = (
        "✈️ <b>Поиск билетов на Авиасейлс</b>\n\n"
        f"✈️ Вылет: <b>{fn}</b>\n\n"
        "<b>Шаг 2 из 3:</b> Выберите <b>город прибытия (Куда)</b> 📍"
    )
    keyboard = []
    row = []
    for name, code in POPULAR_TO_CITIES:
        row.append({"text": name, "callback_data": f"wizard_to:{code}:{name}"})
        if len(row) == 2:
            keyboard.append(row)
            row = []
    if row:
        keyboard.append(row)
    keyboard.append([{"text": "✎ Ввести другой город вручную", "callback_data":
"wizard_type:to"}])
    keyboard.append([{"text": "⬅️ ВАК", "callback_data": "wizard_back:1"}])
    return text, {"inline_keyboard": keyboard}

def step3_calendar_panel(sess, year=None, month=None):
    fn = sess.get('from_name', 'Откуда')
    tn = sess.get('to_name', 'Куда')
    c1 = sess.get("from_code")
    c2 = sess.get("to_code")
    now = datetime.now()
    if year is None or month is None:
        year, month = now.year, now.month
    sess['cal_year'] = year
    sess['cal_month'] = month
    time_filter = sess.get('time_filter', 'any')
    time_info = TIME_FILTERS.get(time_filter, TIME_FILTERS['any'])
    month_str = f"{year}-{month:02d}"

```

```

cached_tickets = None
if c1 and c2:
    cache_key = (c1, c2, month_str)
    if cache_key in PRICES_CACHE:
        ts, data = PRICES_CACHE[cache_key]
        if time.time() - ts < CACHE_TTL:
            cached_tickets = data
    else:
        threading.Thread(target=get_official_aviasales_prices, args=(c1, c2, month_str),
daemon=True).start()
    min_prices = {}
    if cached_tickets:
        for t in cached_tickets:
            dep = t.get('departure_at', "")
            if len(dep) >= 10 and dep.startswith(month_str):
                try:
                    d_day = int(dep[8:10])
                    if time_filter != "any" and "T" in dep:
                        hour = int(dep[11:13])
                        h_start, h_end = time_info["hours"]
                        if not (h_start <= hour < h_end):
                            continue
                    p = t.get("price", 0)
                    if p > 0:
                        if d_day not in min_prices or p < min_prices[d_day]:
                            min_prices[d_day] = p
                except Exception:
                    pass
    text = (
        "✈️ <b>Поиск билетов на Авиасейлс</b>\n\n"
        f"📍 Маршрут: <b>{fn} → {tn}</b>\n"
        f"🕒 Вылет: <b>{time_info['icon']} {time_info['label']}</b>\n\n"
        f"<b>Шаг 3 из 3:</b> Выберите <b>дату вылета</b> ({MONTHS_RU[month]} {year}) или
период 🗓️"
    )
    if min_prices:
        cheapest_sample = sorted(min_prices.items(), key=lambda pair: pair[-1])[4]
        price_tips = [f"{d:02d} {MONTHS_RU[month][:3].lower()} ({p:,.0f} ₺)" for d, p in
cheapest_sample]
        joined_tips = ", ".join(price_tips)
        text += f"\n💰 <i>Выгодные дни: {joined_tips}</i>"
    keyboard = []
    # Листание месяцев

```

```

keyboard.append([
    {"text": "◀", "callback_data": f"wiz_cal_prev:{year}:{month}"},
    {"text": f"{MONTHS_RU[month]} {year}", "callback_data": "ignore"},
    {"text": "▶", "callback_data": f"wiz_cal_next:{year}:{month}"},
])
# Дни недели
keyboard.append([{"text": d, "callback_data": "ignore"} for d in ["Пн", "Вт", "Ср", "Чт", "Пт",
"Сб", "Вс"]])
# Сетка дней календаря
today = now.date()
cal = calendar.monthcalendar(year, month)
for week in cal:
    row = []
    for day in week:
        if day == 0:
            row.append({"text": " ", "callback_data": "ignore"})
        else:
            day_date = datetime(year, month, day).date()
            if day_date < today:
                row.append({"text": ".", "callback_data": "ignore"})
            else:
                row.append({"text": str(day), "callback_data":
f"wiz_cal_day:{year}:{month:02d}:{day:02d}"})
    keyboard.append(row)
# Выбор периода времени суток
tf_row = []
for k in ["any", "morning", "day", "evening"]:
    tf = TIME_FILTERS[k]
    active = "✅ " if k == time_filter else ""
    tf_row.append({"text": f"{active}{tf['icon']} {tf['short']}", "callback_data": f"wiz_time:{k}"})
keyboard.append(tf_row)
# Выбор периода поездки
keyboard.append([
    {"text": "☂ Ближайшие вых-е", "callback_data": "wiz_period:weekend"},
    {"text": "📅 На неделю", "callback_data": "wiz_period:week"},
])
keyboard.append([
    {"text": "📅 На 2 недели", "callback_data": "wiz_period:2weeks"},
    {"text": f"📅 Лучшее за {MONTHS_RU[month]}", "callback_data":
f"wiz_period:month:{year}:{month:02d}"},
])
# Ввод вручную и назад
keyboard.append([{"text": "✎ Ввести дату вручную (напр. 15.10)", "callback_data":

```

```

"wizard_type:date"))
    keyboard.append([{"text": "⬅️ ИЗМЕНИТЬ город прибытия", "callback_data":
"wizard_back:2"}])
    return text, {"inline_keyboard": keyboard}

def show_ticket_card(chat_id, sess, target_date_iso, msg_id=None):
    c1, name1 = sess['from_code'], sess['from_name']
    c2, name2 = sess['to_code'], sess['to_name']
    time_filter = sess.get('time_filter', 'any')
    time_info = TIME_FILTERS.get(time_filter, TIME_FILTERS['any'])
    month_str = target_date_iso[:7]
    tickets = get_official_aviasales_prices(c1, c2, month_str)
    day_tickets = [t for t in tickets if t.get('departure_at', '').startswith(target_date_iso)]
    found = None
    time_mismatch = False
    if day_tickets:
        if time_filter != "any":
            h_start, h_end = time_info['hours']
            for t in day_tickets:
                dep = t.get('departure_at', '')
                if "T" in dep:
                    hour = int(dep[11:13])
                    if h_start <= hour < h_end:
                        found = t
                        break
            if not found:
                found = day_tickets[0]
                time_mismatch = True
        else:
            found = day_tickets[0]
    if found:
        price = found.get("price", 0)
        air_code = found.get('airline', '')
        f_num = found.get("flight_number")
        flight_label = get_airline_name(air_code, f_num)
        pretty_date, date_code, dep_time = format_datetime_ru(found.get("departure_at",
target_date_iso))
        time_line = f"⌚ Вылет: <b>{dep_time}</b>" if dep_time else ""
        btn_time = f"(вылет в {dep_time})" if dep_time else ""
        tr_count = found.get("transfers", 0)
        transfers = "Прямой рейс" if tr_count == 0 else f"Пересадок: {tr_count}"
        flight_link = f"https://www.aviasales.ru{found.get('link', '')}"
        time_notice = f"ℹ️ На выбранное время суток ({time_info['label']}) рейсов нет в

```

```

кеше, показан другой рейс за день.</i>\n" if time_mismatch else ""
text = (
    f"📅 <b>Билет на {pretty_date}</b>\n"
    f"✈️ <b>{name1}</b> → ✈️ <b>{name2}</b>\n\n"
    f"💰 <b>Цена: {price:,.0f} ₺</b>{time_line}\n"
    f"🔄 {transfers} | {flight_label}\n"
    f"{time_notice}"
)
markup = {
    "inline_keyboard": [
        [{"text": f"✈️ Купить билет за {price:,.0f} ₺{btn_time}", "url": flight_link}],
        [{"text": "🔔 Следить за ценой (уведомить о скидке)", "callback_data":
f"track:{c1}:{c2}:{target_date_iso}:{price}"}],
        [{"text": "📅 17 Выбрать другую дату", "callback_data": "wizard_back:3"}, {"text": "🔄
Новый поиск", "callback_data": "wizard_restart"}]
    ]
}
elif tickets:
    best = tickets[0]
    b_price = best.get("price", 0)
    b_date, _, b_time = format_datetime_ru(best.get("departure_at"))
    air_code = best.get('airline', "")
    f_num = best.get("flight_number")
    air_name = get_airline_name(air_code, f_num)
    flight_link = f"https://www.aviasales.ru{best.get('link', '')}"
    time_info_str = f" в {b_time}" if b_time else ""
    pretty_target, _, _ = format_datetime_ru(target_date_iso)
    text = (
        f"📅 <b>Направление: {name1} → {name2}</b>\n"
        f"📅 17 Выбранный день: <b>{pretty_target}</b> (на этот день нет билетов в кеше)\n\n"
        f"💰 <b>Самая выгодная цена рядом: {b_price:,.0f} ₺</b>\n"
        f"📅 17 Дата вылета: <b>{b_date}</b>{time_info_str} ({air_name})\n"
    )
    markup = {
        "inline_keyboard": [
            [{"text": f"✈️ Купить за {b_price:,.0f} ₺ ({b_date})", "url": flight_link}],
            [{"text": "🔔 Следить за ценой на эту дату", "callback_data":
f"track:{c1}:{c2}:{target_date_iso}:{b_price}"}],
            [{"text": "📅 17 Выбрать другую дату", "callback_data": "wizard_back:3"}, {"text": "🔄
Новый поиск", "callback_data": "wizard_restart"}]
        ]
    }
else:

```

```

pretty_target, date_code, _ = format_datetime_ru(target_date_iso)
direct_url = f"https://www.aviasales.ru/search/{c1}{date_code}{c2}1"
text = (
    f"📌 <b>{name1}</b> → {name2}</b> на <b>{pretty_target}</b>\n\n"
    "Нажмите кнопку ниже для онлайн-поиска рейсов 📌"
)
markup = {
    "inline_keyboard": [
        [{"text": "✈️ Проверить билеты на Авиасейлс", "url": direct_url}],
        [{"text": "📅 Выбрать другую дату", "callback_data": "wizard_back:3"}, {"text": "🔄"}
    ]
}
Новый поиск", "callback_data": "wizard_restart"]]
    ]
}
if msg_id:
    res = edit_tg(chat_id, msg_id, text, markup)
    if not res.get("ok"):
        send_tg(chat_id, text, markup)
else:
    send_tg(chat_id, text, markup)

def show_period_ticket_card(chat_id, sess, period_type, param=None, msg_id=None):
    c1, name1 = sess['from_code'], sess['from_name']
    c2, name2 = sess['to_code'], sess['to_name']
    time_filter = sess.get('time_filter', 'any')
    time_info = TIME_FILTERS.get(time_filter, TIME_FILTERS['any'])
    now = datetime.now()
    today = now.date()
    target_dates = []
    period_title = ""
    if period_type == "weekend":
        days_to_fri = (4 - today.weekday()) % 7
        if days_to_fri == 0 and now.hour >= 18:
            days_to_fri = 7
        fri = today + timedelta(days=days_to_fri)
        target_dates = [(fri + timedelta(days=i)).strftime("%Y-%m-%d") for i in range(3)]
        period_title = "Ближайшие выходные"
    elif period_type == "week":
        target_dates = [(today + timedelta(days=i)).strftime("%Y-%m-%d") for i in range(1, 8)]
        period_title = "Ближайшая неделя (7 дней)"
    elif period_type == "2weeks":
        target_dates = [(today + timedelta(days=i)).strftime("%Y-%m-%d") for i in range(1, 15)]
        period_title = "Ближайшие 2 недели"
    elif period_type == "month":

```

```

if param:
    y_str, m_str = param.split(":")
    y, m = int(y_str), int(m_str)
else:
    y, m = now.year, now.month
period_title = f"{MONTHS_RU[m]} {y}"
_, last_day = calendar.monthrange(y, m)
for d in range(1, last_day + 1):
    d_obj = datetime(y, m, d).date()
    if d_obj >= today:
        target_dates.append(d_obj.strftime("%Y-%m-%d"))
months_needed = sorted(list(set(d[:7] for d in target_dates)))
all_tickets = []
for m_str in months_needed:
    all_tickets.extend(get_official_aviasales_prices(c1, c2, m_str))
period_tickets = [
    t for t in all_tickets
    if any(t.get('departure_at', "").startswith(d) for d in target_dates)
]
time_mismatch = False
if period_tickets and time_filter != "any":
    h_start, h_end = time_info["hours"]
    matching_time = []
    for t in period_tickets:
        dep = t.get('departure_at', "")
        if "T" in dep:
            hour = int(dep[11:13])
            if h_start <= hour < h_end:
                matching_time.append(t)
    if matching_time:
        period_tickets = matching_time
    else:
        time_mismatch = True
if period_tickets:
    period_tickets.sort(key=lambda t: t.get("price", float("inf")))
    best = period_tickets[0]
    price = best.get("price", 0)
    dep_full = best.get('departure_at', "")
    pretty_date, date_code, dep_time = format_datetime_ru(dep_full)
    air_code = best.get('airline', "")
    f_num = best.get("flight_number")
    flight_label = get_airline_name(air_code, f_num)
    tr_count = best.get("transfers", 0)

```

```

transfers = "Прямой рейс" if tr_count == 0 else f"Пересадок: {tr_count}"
flight_link = f"https://www.aviasales.ru{best.get('link', '')}"
time_line = f" • 🕒 Вылет: <b>{dep_time}</b>" if dep_time else ""
btn_time = f" (вылет в {dep_time})" if dep_time else ""
time_notice = f"\n<i>🔔 На выбранный период времени ({time_info['label']}) рейсов нет в кеше, показан лучший рейс за период.</i>\n" if time_mismatch else ""
text = (
    f"🏆 <b>Лучший билет на период: {period_title}</b>\n"
    f"✈️ <b>{name1}</b> → ✈️ <b>{name2}</b>\n\n"
    f"📅 Дата вылета: <b>{pretty_date}</b>{time_line}\n"
    f"💰 <b>Цена: {price:,.0f} ₽</b>\n"
    f"✈️ {transfers} | {flight_label}\n"
    f"{time_notice}"
)
markup = {
    "inline_keyboard": [
        [{"text": f"✈️ Купить билет за {price:,.0f} ₽{btn_time}", "url": flight_link}],
        [{"text": "🔔 Следить за ценой", "callback_data":
f"track:{c1}:{c2}:{dep_full[:10]}:{price}"}],
        [{"text": "📅 Выбрать другую дату", "callback_data": "wizard_back:3"}, {"text": "🔄
Новый поиск", "callback_data": "wizard_restart"}]
    ]
}
else:
    direct_url = f"https://www.aviasales.ru/search/{c1}{c2}1"
    text = (
        f"🏆 <b>Направление: {name1} → {name2}</b>\n"
        f"📅 Период: <b>{period_title}</b>\n\n"
        "В кеше пока нет билетов на этот период. Нажмите кнопку ниже для онлайн-поиска всех доступных рейсов 🗨"
    )
    markup = {
        "inline_keyboard": [
            [{"text": "✈️ Проверить билеты на Авиасейлс", "url": direct_url}],
            [{"text": "📅 Выбрать другую дату", "callback_data": "wizard_back:3"}, {"text": "🔄
Новый поиск", "callback_data": "wizard_restart"}]
        ]
    }
    if msg_id:
        res = edit_tg(chat_id, msg_id, text, markup)
        if not res.get("ok"):
            send_tg(chat_id, text, markup)
    else:

```

```
send_tg(chat_id, text, markup)
```

```
# ===== ФОНОВЫЙ МОНИТОРИНГ =====
def monitor_worker():
    while True:
        time.sleep(180)
        try:
            subs = load_subs()
            if not subs:
                continue
            updated = False
            for sub in subs:
                chat_id = sub['chat_id']
                o, d, dt, pr = sub['origin'], sub['dest'], sub['date'], sub['price']
                o_name = sub.get("origin_name") or get_city_name(o)
                d_name = sub.get("dest_name") or get_city_name(d)
                tickets = get_official_aviasales_prices(o, d, dt[:7])
                for t in tickets:
                    if t.get('departure_at', "").startswith(dt):
                        curr = t.get("price", 0)
                        if 0 < curr < pr:
                            diff = pr - curr
                            flight_link = f"https://www.aviasales.ru{t.get('link', '')}"
                            pretty, _, dep_time = format_datetime_ru(t.get("departure_at"))
                            air_code = t.get('airline', "")
                            f_num = t.get("flight_number")
                            air_label = get_airline_name(air_code, f_num)
                            time_info = f" (вылет в {dep_time})" if dep_time else ""
                            msg = (
                                f"🔥 <b>ЦЕНА СНИЗИЛАСЬ!</b>\n\n"
                                f"✈️ <b>{o_name} → {d_name}</b>\n"
                                f"📅 <b>{pretty}</b>{time_info}\n"
                                f"✈️ <b>{air_label}</b>\n"
                                f"Было: <s>{pr:,.0f} ₹</s>\n"
                                f"Стало: <b>{curr:,.0f} ₹</b> (скидка: {diff:,.0f} ₹!)"
                            )
                            send_tg(chat_id, msg, {"inline_keyboard": [[{"text": f"✈️ Купить билет за {curr:,.0f} ₹", "url": flight_link}]]})
                            sub['price'] = curr
                            updated = True
                            break
            if updated:
                save_subs(subs)
```

```
except Exception:
    pass
```

```
# ===== ОБРАБОТКА CALLBACK-КНОПОК =====
```

```
def handle_callback(cq):
    cq_id = cq['id']
    data = cq.get('data', "")
    chat_id = cq['message']['chat']['id']
    msg_id = cq['message']['message_id']
    sess = get_session(chat_id)
    # Игнор
    if data == "ignore":
        answer_cq(cq_id)
        return
    # Начать заново
    if data == "wizard_restart":
        answer_cq(cq_id)
        sess['from_code'] = None
        sess['from_name'] = None
        sess['to_code'] = None
        sess['to_name'] = None
        sess['time_filter'] = "any"
        sess['mode'] = "wizard_from"
        text, markup = step1_from_panel()
        edit_tg(chat_id, msg_id, text, markup)
        return
    # Возврат по шагам
    if data.startswith("wizard_back:"):
        answer_cq(cq_id)
        _, step = data.split(":", 1)
        if step == "1":
            sess['mode'] = "wizard_from"
            text, markup = step1_from_panel()
            edit_tg(chat_id, msg_id, text, markup)
        elif step == "2":
            sess['mode'] = "wizard_to"
            text, markup = step2_to_panel(sess)
            edit_tg(chat_id, msg_id, text, markup)
        elif step == "3":
            sess['mode'] = "wizard_calendar"
            y = sess.get("cal_year")
            m = sess.get("cal_month")
            text, markup = step3_calendar_panel(sess, y, m)
```

```

        edit_tg(chat_id, msg_id, text, markup)
    return

# Шаг 1: Выбор города вылета
if data.startswith("wizard_from:"):
    answer_cq(cq_id)
    _, code, name = data.split(":", 2)
    sess['from_code'] = code
    sess['from_name'] = name
    sess['mode'] = "wizard_to"
    text, markup = step2_to_panel(sess)
    edit_tg(chat_id, msg_id, text, markup)
    return

# Шаг 2: Выбор города прибытия
if data.startswith("wizard_to:"):
    answer_cq(cq_id)
    _, code, name = data.split(":", 2)
    sess['to_code'] = code
    sess['to_name'] = name
    sess['mode'] = "wizard_calendar"
    text, markup = step3_calendar_panel(sess)
    edit_tg(chat_id, msg_id, text, markup)
    return

# Ввод вручную
if data.startswith("wizard_type:"):
    answer_cq(cq_id)
    _, target = data.split(":", 1)
    if target == "from":
        sess['mode'] = "type_from"
        text = "✎ Напишите <b>город вылета</b> в чат (например: <i>Казань</i>,"
        <i>Тюмень</i>)."
        markup = {"inline_keyboard": [{"text": "⬅️ Отмена (к списку городов)", "callback_data":
"wizard_back:1"}]}}
    elif target == "to":
        sess['mode'] = "type_to"
        text = "✎ Напишите <b>город прибытия</b> в чат (например: <i>Сочи</i>,"
        <i>Дубай</i>)."
        markup = {"inline_keyboard": [{"text": "⬅️ Отмена (к списку городов)", "callback_data":
"wizard_back:2"}]}}
    else:
        sess['mode'] = "type_date"
        text = "✎ Напишите <b>дату вылета</b> в чат (например: <code>15.10</code> или
<code>20 ноября</code>)."
        markup = {"inline_keyboard": [{"text": "⬅️ Отмена (в календарь)", "callback_data":

```

```

"wizard_back:3"]]]}
    edit_tg(chat_id, msg_id, text, markup)
    return
# Выбор времени суток вылета
if data.startswith("wiz_time:"):
    answer_cq(cq_id)
    _, t_val = data.split(":", 1)
    sess['time_filter'] = t_val
    y = sess.get("cal_year", datetime.now().year)
    m = sess.get("cal_month", datetime.now().month)
    text, markup = step3_calendar_panel(sess, y, m)
    edit_tg(chat_id, msg_id, text, markup)
    return
# Выбор периода поездки
if data.startswith("wiz_period:"):
    answer_cq(cq_id)
    _, p_type, *rest = data.split(":")
    param = ":".join(rest) if rest else None
    sess['mode'] = "main"
    show_period_ticket_card(chat_id, sess, p_type, param=param, msg_id=msg_id)
    return
# Шаг 3: Листание месяцев в календаре
if data.startswith("wiz_cal_prev:") or data.startswith("wiz_cal_next:"):
    _, y_str, m_str = data.split(":")
    y, m = int(y_str), int(m_str)
    today = datetime.now().date()
    if "prev" in data:
        m -= 1
        if m < 1:
            m = 12
            y -= 1
        if y < today.year or (y == today.year and m < today.month):
            answer_cq(cq_id, "Нельзя выбрать прошедший месяц")
            return
    else:
        m += 1
        if m > 12:
            m = 1
            y += 1
    answer_cq(cq_id)
    sess['cal_year'] = y
    sess['cal_month'] = m
    text, markup = step3_calendar_panel(sess, y, m)

```

```

    edit_tg(chat_id, msg_id, text, markup)
    return
# Шаг 3: Выбор дня в календаре
if data.startswith("wiz_cal_day:"):
    answer_cq(cq_id)
    _, y, m, d = data.split(".")
    target_iso = f"{y}-{m}-{d}"
    sess['mode'] = "main"
    show_ticket_card(chat_id, sess, target_iso, msg_id=msg_id)
    return
# Быстрые даты (совместимость)
if data == "wiz_weekend":
    answer_cq(cq_id)
    sess['mode'] = "main"
    show_period_ticket_card(chat_id, sess, "weekend", msg_id=msg_id)
    return
if data == "wiz_week":
    answer_cq(cq_id)
    sess['mode'] = "main"
    show_period_ticket_card(chat_id, sess, "week", msg_id=msg_id)
    return
# Подписка на рейс
if data.startswith("track:"):
    _, o, d, dt, pr_str = data.split(".")
    pr = int(pr_str)
    subs = load_subs()
    already = any(
        str(s.get("chat_id")) == str(chat_id)
        and s.get("origin") == o
        and s.get("dest") == d
        and s.get("date") == dt
        for s in subs
    )
    o_name = sess.get("from_name") or get_city_name(o)
    d_name = sess.get("to_name") or get_city_name(d)
    if not already:
        subs.append({
            "chat_id": str(chat_id),
            "origin": o,
            "dest": d,
            "origin_name": o_name,
            "dest_name": d_name,
            "date": dt,

```

```

        "price": pr,
    })
    save_subs(subs)
    answer_cq(cq_id, "✅ Подписка сохранена!")
    pretty_d, _, _ = format_datetime_ru(dt)
    send_tg(
        chat_id,
        f"🔔 <b>Мониторинг цены включен!</b>\n"
        f"Рейс: <b>{o_name} → {d_name}</b>\n"
        f"📅 <b>{pretty_d}</b>\n"
        f"💰 Текущая цена: <b>{pr:,.0f} ₽</b>\n\n"
        "Бот оповестит вас в чате, как только цена снизится! 📈"
    )
else:
    answer_cq(cq_id, "Вы уже следите за этим рейсом!")
    return
# Удаление подписки
if data.startswith("del:"):
    _, o, d, dt = data.split(":")
    subs = [
        s for s in load_subs()
        if not (
            str(s.get("chat_id")) == str(chat_id)
            and s.get("origin") == o
            and s.get("dest") == d
            and s.get("date") == dt
        )
    ]
    save_subs(subs)
    answer_cq(cq_id, "Удалено!")
    send_tg(chat_id, f"❌ Отслеживание {o} → {d} на {dt} удалено.")
    return

```

===== ОБРАБОТКА ТЕКСТОВЫХ СООБЩЕНИЙ
=====

```

def handle_text_message(msg):
    chat_id = msg['chat']['id']
    raw_text = msg.get('text', "").strip()
    if not raw_text:
        return
    sess = get_session(chat_id)
    # Старт / новый поиск
    if raw_text.startswith("/start") or "новый поиск" in raw_text.lower() or "начать" in

```

```

raw_text.lower():
    sess['from_code'] = None
    sess['from_name'] = None
    sess['to_code'] = None
    sess['to_name'] = None
    sess['time_filter'] = "any"
    sess['mode'] = "wizard_from"
    text, markup = step1_from_panel()
    send_tg(chat_id, "✈️ <b>Поиск дешевых авиабилетов</b>", main_reply_keyboard())
    send_tg(chat_id, text, markup)
    return
# Сброс
if "сбросить" in raw_text.lower():
    sess['from_code'] = None
    sess['from_name'] = None
    sess['to_code'] = None
    sess['to_name'] = None
    sess['time_filter'] = "any"
    sess['mode'] = "wizard_from"
    text, markup = step1_from_panel()
    send_tg(chat_id, "🔄 Выбор маршрута очищен.", main_reply_keyboard())
    send_tg(chat_id, text, markup)
    return
# ТОП-5 дешевых
if "топ" in raw_text.lower():
    c1, name1 = sess.get("from_code"), sess.get("from_name")
    c2, name2 = sess.get("to_code"), sess.get("to_name")
    if not c1 or not c2:
        send_tg(
            chat_id,
            "⚠️ <b>Сначала завершите выбор маршрута!</b>\n"
            "Выберите город вылета и прибытия в интерактивном сообщении выше.",
            main_reply_keyboard()
        )
        return
    tickets = get_official_aviasales_prices(c1, c2)
    if tickets:
        text = f"🔥 <b>ТОП-5 самых дешевых билетов ({name1} → {name2}):</b>\n\n"
        buttons = []
        for i, t in enumerate(tickets[:5], 1):
            price = t.get("price", 0)
            pretty, _, dep_time = format_datetime_ru(t.get("departure_at"))
            air_code = t.get('airline', '')

```

```

    f_num = t.get("flight_number")
    air_name = get_airline_name(air_code, f_num)
    link = f"https://www.aviasales.ru{t.get('link', '')}"
    text += f"{i}. <b>{price:,.0f} ₽</b> — {pretty} ({air_name})\n"
    buttons.append([{"text": f"✈️ {i}. Купить за {price:,.0f} ₽ ({pretty[:6]})", "url": link}])
    send_tg(chat_id, text, {"inline_keyboard": buttons})
else:
    send_tg(chat_id, f"Билеты по направлению {name1} → {name2} не найдены.")
return

# Мои отслеживания
if "отслеживан" in raw_text.lower() or raw_text.startswith("/my"):
    subs = [s for s in load_subs() if str(s.get("chat_id")) == str(chat_id)]
    if not subs:
        send_tg(
            chat_id,
            "📋 <b>У вас пока нет активных отслеживаний.</b>\n\n"
            "При поиске билетов нажимайте «Следить за ценой», и бот оповестит о скидке.",
            main_reply_keyboard()
        )
    return
for s in subs:
    pretty, _, _ = format_datetime_ru(s['date'])
    o_name = s.get("origin_name") or get_city_name(s['origin'])
    d_name = s.get("dest_name") or get_city_name(s['dest'])
    markup = {
        "inline_keyboard": [[{
            "text": "❌ Удалить отслеживание",
            "callback_data": f"del:{s['origin']}:{s['dest']}:{s['date']}"
        }]]
    }
    send_tg(chat_id, f"📍 <b>{o_name} → {d_name}</b>\n📅 {pretty}\n💰 Цена:
<b>{s['price']:,.0f} ₽</b>", markup)
    return

# Ручной ввод города вылета
if sess.get("mode") == "type_from":
    c_code, c_name = find_city_smart(raw_text)
    if c_code:
        sess['from_code'] = c_code
        sess['from_name'] = c_name
        sess['mode'] = "wizard_to"
        text, markup = step2_to_panel(sess)
        send_tg(chat_id, text, markup)
    else:

```

```
        send_tg(chat_id, f"Город «{raw_text}» не найден. Попробуйте написать иначе  
(например: <i>Москва</i>):")
```

```
    return
```

```
    # Ручной ввод города прибытия
```

```
    if sess.get("mode") == "type_to":
```

```
        c_code, c_name = find_city_smart(raw_text)
```

```
        if c_code:
```

```
            sess['to_code'] = c_code
```

```
            sess['to_name'] = c_name
```

```
            sess['mode'] = "wizard_calendar"
```

```
            text, markup = step3_calendar_panel(sess)
```

```
            send_tg(chat_id, text, markup)
```

```
        else:
```

```
            send_tg(chat_id, f"Город «{raw_text}» не найден. Попробуйте написать иначе  
(например: <i>Сочи</i>):")
```

```
    return
```

```
    # Ручной ввод даты
```

```
    if sess.get("mode") == "type_date":
```

```
        t_iso, _ = extract_date(raw_text)
```

```
        if t_iso:
```

```
            sess['mode'] = "main"
```

```
            show_ticket_card(chat_id, sess, t_iso)
```

```
        else:
```

```
            send_tg(chat_id, "Не удалось распознать дату. Напишите, например:  
<code>15.10</code> или <code>20 ноября</code>:")
```

```
    return
```

```
    # Дефолтный ответ
```

```
    text, markup = step1_from_panel()
```

```
    send_tg(chat_id, text, markup)
```

```
# ===== ОСНОВНОЙ ЦИКЛ БОТА =====
```

```
def main():
```

```
    threading.Thread(target=monitor_worker, daemon=True).start()
```

```
    print(">>> Бот запущен (максимальная скорость отклика) <<<")
```

```
    offset = 0
```

```
    poll_session = requests.Session()
```

```
    poll_session.mount("https://", adapter)
```

```
    while True:
```

```
        try:
```

```
            res = poll_session.get(f"{URL}/getUpdates?offset={offset}&timeout=20",  
timeout=25).json()
```

```
            if not res.get("ok"):
```

```
                time.sleep(1)
```

```
        continue
    for item in res.get("result", []):
        offset = item['update_id'] + 1
        if "callback_query" in item:
            threading.Thread(target=handle_callback, args=(item['callback_query'],),
daemon=True).start()
            elif "message" in item and "text" in item['message']:
                threading.Thread(target=handle_text_message, args=(item['message'],),
daemon=True).start()
        except Exception as e:
            print("Ошибка цикла:", e)
            time.sleep(1)

if __name__ == "__main__":
    main()
```